SPRINT 4
Tasca S4.01. Creació de Base de Dades

Nivell 1

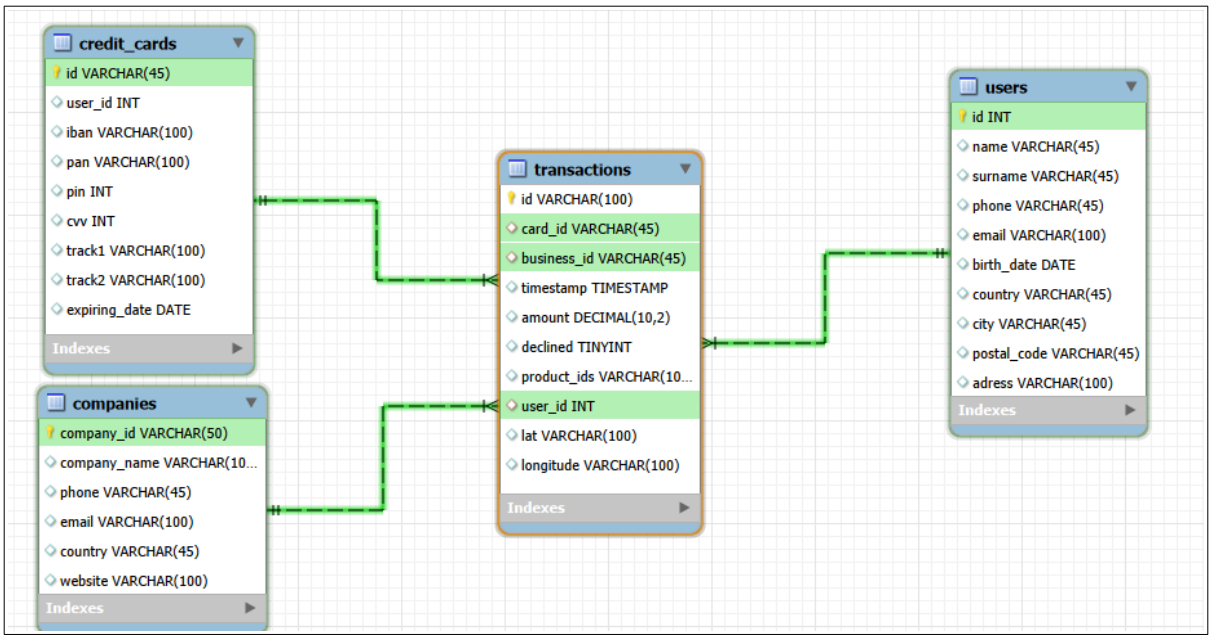
Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Se diseña un modelo en estrella donde la tabla transactions es la tabla de hechos. Las tablas credit_cards, companies y users (se unifican las dos tablas de usuarios en una única tabla) son dimensiones que aportan información descriptiva a las transacciones.
Las PK de las tablas son:

- transactions (id)
- credit_cards (id)
- companies (company_id)
- users (id)

La relación de las tablas se realiza mediante la tabla de hechos, transactions y sería la siguiente:

- La relación entre credit_cards y transactions es de uno a muchos (1:N), donde una tarjeta puede tener varias transacciones, pero cada transacción pertenece a una sola tarjeta. Esta relación se establece mediante la clave foránea card_id.
- La relación entre companies y transactions es de uno a muchos (1:N), donde una compañía puede tener varias transacciones, pero cada transacción pertenece a una sola compañía. Esta relación se establece mediante la clave foránea business_id.
- La relación entre users y transactions es de uno a muchos (1:N), donde un usuario puede tener varias transacciones, pero cada transacción pertenece a un solo usuario. Esta relación se establece mediante la clave foránea user_id.



A continuación se crea la base de datos y se insertan las tablas. Inicialmente, todos los campos se definen como VARCHAR, posteriormente se modificaran a los adecuados. En transactions se toma la decisión de cambiar el nombre de business_id por company_id para mantener la nomenclatura para el mismo campo.

Para la creación de la tabla users, se decide de unificar los dos archivos csv (american_users y european_users). Para no perder la información de la región se añade una columna con dicho dato.

A continuación se insertan los datos en cada tabla. En la tabla credit_cards se modifica el campo expiring_date para poder modificar después a DATE.
En la tabla users se modifica el campo birth_date y se añade la región.

```sql
CREATE DATABASE IF NOT EXISTS Ventas;
USE ventas;
CREATE TABLE transactions (
    id VARCHAR (250),
    card_id VARCHAR (250),
    company_id VARCHAR (250),
    timestamp VARCHAR (250),
    amount VARCHAR (250),
    declined VARCHAR (250),
    product_ids VARCHAR (250),
    user_id VARCHAR (250),
    lat VARCHAR (250),
    longitude VARCHAR (250)
  );
CREATE TABLE credit_cards (
    id VARCHAR (250),
    user_id VARCHAR (250),
    iban VARCHAR (250),
    pan VARCHAR (250),
    pin VARCHAR (250),
    cvv VARCHAR (250),
    track1 VARCHAR (250),
    track2 VARCHAR (250),
    expiring_date VARCHAR (250)
  );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:04:07	CREATE DATABASE IF NOT EXISTS Ventas	1 row(s) affected	0.016 sec
2	19:04:09	USE ventas	0 row(s) affected	0.000 sec
3	19:04:11	CREATE TABLE transactions (id VARCHAR (250), card_id VARCHAR (250), company_id VARCHAR (250), timestamp VARCHAR (250), am...	0 row(s) affected	0.078 sec
4	19:04:14	CREATE TABLE credit_cards (id VARCHAR (250), user_id VARCHAR (250), iban VARCHAR (250), pan VARCHAR (250), pin VARCHAR (2...	0 row(s) affected	0.062 sec

```sql
CREATE TABLE companies (
    company_id VARCHAR (250),
    company_name VARCHAR (250),
    phone VARCHAR (250),
    email VARCHAR (250),
    country VARCHAR (250),
    website VARCHAR (250)
  );
CREATE TABLE users (
    id VARCHAR (250),
    name VARCHAR (250),
    surname VARCHAR (250),
    phone VARCHAR (250),
    email VARCHAR (250),
    birth_date VARCHAR (250),
    country VARCHAR (250),
    city VARCHAR (250),
    postal_code VARCHAR (250),
    adress VARCHAR (250),
    region VARCHAR (250)
  );
```

Output

Action Output

#	Time	Action	Message	Duration / Fet
1	18:59:33	CREATE TABLE companies (company_id VARCHAR (250), company_name VARCHAR (250), phone VARCHAR (250), email VARCHAR (250),...	0 row(s) affected	0.062 sec
2	18:59:35	CREATE TABLE users (id VARCHAR (250), name VARCHAR (250), surname VARCHAR (250), phone VARCHAR (250), email VARCHAR (2...	0 row(s) affected	0.078 sec

```sql
LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
  (id, user_id, iban, pan, pin, cvv, track1, track2, @expiring_date)
SET
    expiring_date = STR_TO_DATE(@expiring_date, '%m/%d/%y');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:00:46	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\companies.csv' INTO TABLE companies FIELDS TER...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec
2	19:00:48	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\credit_cards.csv' INTO TABLE credit_cards FIELDS T...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.328 sec

```sql
LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, @birth_date, country, city, postal_code, adress)
SET
    birth_date = STR_TO_DATE(@birth_date,'%b %d, %Y'),
    region = 'europa';

LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, @birth_date, country, city, postal_code, adress)
SET
    birth_date = STR_TO_DATE(@birth_date,'%b %d, %Y'),
    region = 'america';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:01:59	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\european_users.csv' INTO TABLE users FIELDS TER...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.297 sec
2	19:02:01	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\american_users.csv' INTO TABLE users FIELDS TER...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec

```sql
LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message	Duration / Fet
1	19:05:35	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\transactions.csv' INTO TABLE transactions FIELDS T...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	2.375 sec

Una de las dudas encontradas ha sido cómo determinar la longitud adecuada de los campos VARCHAR sin tener revisar manualmente todos los valores. Para ello, se ejecuta una consulta que permite identificar la longitud máxima de cada columna y la presencia de valores nulos. Esto facilita acotar el tamaño de los campos. Este proceso se realiza para cada tabla.

```sql
SELECT
    CONCAT(
        'MAX(CHAR_LENGTH(', COLUMN_NAME, ')) AS ', COLUMN_NAME, '_len, ',
        'SUM(CASE WHEN ', COLUMN_NAME, ' IS NULL OR ', COLUMN_NAME, ' = \'\' THEN 1 ELSE 0 END) AS ', COLUMN_NAME, '_nulls',','
    ) AS query_fragment
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'companies'
  AND TABLE_SCHEMA = 'ventas';
SELECT
MAX(CHAR_LENGTH(company_id)) AS company_id_len, SUM(CASE WHEN company_id IS NULL OR company_id = '' THEN 1 ELSE 0 END) AS company_id_nulls,
MAX(CHAR_LENGTH(company_name)) AS company_name_len, SUM(CASE WHEN company_name IS NULL OR company_name = '' THEN 1 ELSE 0 END) AS company_name_nulls,
MAX(CHAR_LENGTH(phone)) AS phone_len, SUM(CASE WHEN phone IS NULL OR phone = '' THEN 1 ELSE 0 END) AS phone_nulls,
MAX(CHAR_LENGTH(email)) AS email_len, SUM(CASE WHEN email IS NULL OR email = '' THEN 1 ELSE 0 END) AS email_nulls,
MAX(CHAR_LENGTH(country)) AS country_len, SUM(CASE WHEN country IS NULL OR country = '' THEN 1 ELSE 0 END) AS country_nulls,
MAX(CHAR_LENGTH(website)) AS website_len, SUM(CASE WHEN website IS NULL OR website = '' THEN 1 ELSE 0 END) AS website_nulls
FROM companies;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id_len	company_id_nulls	company_name_len	company_name_nulls	phone_len	phone_nulls	email_len	email_nulls	country_len	country_nulls	website_len	website_nulls
6	0	35	0	14	0	38	0	14	0	32	0

Result 55 | Read On

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	11:55:38	SELECT CONCAT('MAX(CHAR_LENGTH(', COLUMN_NAME, ')) AS ', COLUMN_NAME, '_len, ', ...	6 row(s) returned	0.016 sec / 0.000 se
2	11:55:41	SELECT MAX(CHAR_LENGTH(company_id)) AS company_id_len, SUM(CASE WHEN company_id IS NULL...	1 row(s) returned	0.000 sec / 0.000 se

A continuación, se realizan las modificaciones correspondientes en las tablas, se añaden el primary key y los foreign key. Se consideran los campos numéricos como varchar si son descriptivos, es decir, no serán utilizados para realizar operaciones, a excepción de users_id y product_id, que se definen como INT para permitir la ordenación.

```sql
ALTER TABLE credit_cards
    MODIFY COLUMN id VARCHAR (10) NOT NULL,
    MODIFY COLUMN user_id INT,
    MODIFY COLUMN iban VARCHAR (50),
    MODIFY COLUMN pan VARCHAR (25),
    MODIFY COLUMN pin VARCHAR (5),
    MODIFY COLUMN cvv VARCHAR (5),
    MODIFY COLUMN track1 VARCHAR (60),
    MODIFY COLUMN track2 VARCHAR (60),
    MODIFY COLUMN expiring_date DATE,
    ADD PRIMARY KEY (id);

ALTER TABLE companies
    MODIFY COLUMN company_id VARCHAR (10) NOT NULL,
    MODIFY COLUMN company_name VARCHAR (50),
    MODIFY COLUMN phone VARCHAR (20),
    MODIFY COLUMN email VARCHAR (50),
    MODIFY COLUMN country VARCHAR (20),
    MODIFY COLUMN website VARCHAR (50),
    ADD PRIMARY KEY (company_id);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:10:32	ALTER TABLE credit_cards MODIFY COLUMN id VARCHAR (10) NOT NULL, MODIFY COLUMN user_id INT, MODIFY COLUMN iban VARCHAR (5...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.563 sec
2	19:10:35	ALTER TABLE companies MODIFY COLUMN company_id VARCHAR (10) NOT NULL, MODIFY COLUMN company_name VARCHAR (50), MODIFY...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.266 sec

```sql
ALTER TABLE users
    MODIFY COLUMN id INT PRIMARY KEY NOT NULL,
    MODIFY COLUMN name VARCHAR (20),
    MODIFY COLUMN surname VARCHAR (20),
    MODIFY COLUMN phone VARCHAR (20),
    MODIFY COLUMN email VARCHAR (50),
    MODIFY COLUMN birth_date DATE,
    MODIFY COLUMN country VARCHAR (20),
    MODIFY COLUMN city VARCHAR (20),
    MODIFY COLUMN postal_code VARCHAR (10),
    MODIFY COLUMN adress VARCHAR (40),
    MODIFY COLUMN region VARCHAR (10);

ALTER TABLE transactions
    MODIFY COLUMN id VARCHAR (45) PRIMARY KEY NOT NULL,
    MODIFY COLUMN card_id VARCHAR (10),
    MODIFY COLUMN company_id VARCHAR (10),
    MODIFY COLUMN timestamp TIMESTAMP,
    MODIFY COLUMN amount DECIMAL (10,2),
    MODIFY COLUMN declined TINYINT,
    MODIFY COLUMN product_ids VARCHAR (50),
    MODIFY COLUMN user_id INT,
    MODIFY COLUMN lat VARCHAR (20),
    MODIFY COLUMN longitude VARCHAR (20),
    ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (card_id) REFERENCES credit_cards(id),
    ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (company_id) REFERENCES companies(company_id),
    ADD CONSTRAINT fk_transactions_users FOREIGN KEY (user_id) REFERENCES users(id);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:11:33	ALTER TABLE users MODIFY COLUMN id INT PRIMARY KEY NOT NULL, MODIFY COLUMN name VARCHAR (20), MODIFY COLUMN surname V...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.453 sec
2	19:11:36	ALTER TABLE transactions MODIFY COLUMN id VARCHAR (45) PRIMARY KEY NOT NULL, MODIFY COLUMN card_id VARCHAR (10), MODIFY C...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	6.250 sec

Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```sql
SELECT id, name, surname
FROM users
WHERE id IN (
    SELECT user_id
    FROM transactions
    WHERE declined = 0
    GROUP BY user_id
    HAVING count(id)>80)
ORDER BY id;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
NULL	NULL	NULL

users 62 ×

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:46:26	SELECT id, name, surname FROM users WHERE id IN (SELECT user_id FROM transactions WHERE ...	3 row(s) returned	0.281 sec / 0.000 sec

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```sql
SELECT iban, ROUND(AVG(amount),2) AS media_importe
FROM credit_cards cc
JOIN transactions t
ON cc.id = t.card_id
JOIN companies c
ON c.company_id = t.company_id
WHERE c.company_name = 'Donec Ltd' AND declined = 0
GROUP BY iban
ORDER BY media_importe DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	media_importe
XX3830178139196201993663 52	680.69
XX6377063573975703949739 13	680.01
XX9713939714652922023122 59	645.46
XX1718471169288923759693 07	628.89
XX2254246388185424062235 75	608.68

Result 64

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:47:44	SELECT iban, ROUND(AVG(amount),2) AS media_importe FROM credit_cards cc JOIN transactions t ON cc....	370 row(s) returned	0.000 sec / 0.000 se

Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Se crea una nueva tabla que clasifica las tarjetas como activas o inactivas en función de sus últimas tres transacciones. Para ello, se utiliza la expresión CASE, que permite crear una nueva columna donde se determina que si la suma de las transacciones declined sea igual a 3, la tarjeta se considera inactiva, en caso contrario, activa.
Además, se utilizan funciones de ventana (window functions), como row_number () para asignar un número a cada transacción dentro de cada tarjeta. Se ordenan las transacciones por fecha (order by timestamp) y se agrupan por tarjeta (partition by card_id), de esta forma, se pueden seleccionar las tres transacciones más recientes de cada tarjeta.

```sql
CREATE TABLE IF NOT EXISTS estado_credit_cards (
    card_id VARCHAR (10) PRIMARY KEY,
    estado VARCHAR (8)
);

INSERT INTO estado_credit_cards (card_id, estado)
SELECT
    card_id,
    CASE
        WHEN SUM(declined) = 3 THEN 'inactiva'
        ELSE 'activa'
    END AS estado
FROM (
    SELECT
        card_id,
        declined,
        ROW_NUMBER() OVER( PARTITION BY card_id ORDER BY timestamp DESC) AS rn
    FROM transactions
    ) t
WHERE rn <= 3
GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:16:01	CREATE TABLE IF NOT EXISTS estado_credit_cards (card_id VARCHAR (10) PRIMARY KEY, estado VARCHAR (8))	0 row(s) affected	0.062 sec
2	19:16:11	INSERT INTO estado_credit_cards (card_id, estado) SELECT card_id, CASE WHEN SUM(declined) = 3 THEN 'inactiva' ELSE 'activa' END ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.531 sec

Exercici 1

Quantes targetes estan actives?

```
SELECT count(card_id) AS total_tarjetas_activas
FROM estado_credit_cards
WHERE estado = 'activa';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_tarjetas_activas
4995

Result 65

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:50:19	SELECT count(card_id) AS total_tarjetas_activas FROM estado_credit_cards WHERE estado = 'activa'	1 row(s) returned	0.000 sec / 0.00

Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids.

Se crea la tabla products, definiendo todos los campos como varchar para luego ajustarlos, como se hizo en las anteriores tablas.
Se insertan los datos teniendo en cuenta que en la columna price aparece el símbolo $ y que en warehouse tenemos valores con '-' y con '--'. Se utilizan variables temporales para limpiar la información antes de insertarla.

```
CREATE TABLE IF NOT EXISTS products (
    id VARCHAR(250),
    product_name VARCHAR(250),
    price VARCHAR(250),
    colour VARCHAR(250),
    weight VARCHAR(250),
    warehouse_id VARCHAR(250)
    );

LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, product_name, @price, colour, weight, @warehouse)
SET
    price = REPLACE(@price, '$', ''),
    warehouse_id = REPLACE(@warehouse, '--', '-');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:15:17	CREATE TABLE IF NOT EXISTS products (id VARCHAR(250), product_name VARCHAR(250), price VA...	0 row(s) affected	0.062 sec
2	13:15:19	LOAD DATA LOCAL INFILE 'C:\\Users\\torre\\OneDrive\\Escritorio\\data temporal\\sprint 4\\products.csv' I...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec

Ejecutamos la consulta para determinar la longitud máxima de cada columna y detectar si hay nulos. Se ejecutan las modificaciones necesarias.

```
SELECT
    MAX(CHAR_LENGTH(id)) AS id_len, SUM(CASE WHEN id IS NULL OR id = '' THEN 1 ELSE 0 END) AS id_nulls,
    MAX(CHAR_LENGTH(product_name)) AS product_name_len, SUM(CASE WHEN product_name IS NULL OR product_name = '' THEN 1 ELSE 0 END) AS product_name_nulls,
    MAX(CHAR_LENGTH(price)) AS price_len, SUM(CASE WHEN price IS NULL OR price = '' THEN 1 ELSE 0 END) AS price_nulls,
    MAX(CHAR_LENGTH(colour)) AS colour_len, SUM(CASE WHEN colour IS NULL OR colour = '' THEN 1 ELSE 0 END) AS colour_nulls,
    MAX(CHAR_LENGTH(weight)) AS weight_len, SUM(CASE WHEN weight IS NULL OR weight = '' THEN 1 ELSE 0 END) AS weight_nulls,
    MAX(CHAR_LENGTH(warehouse_id)) AS warehouse_id_len, SUM(CASE WHEN warehouse_id IS NULL OR warehouse_id = '' THEN 1 ELSE 0 END) AS warehouse_id_nulls
    FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_len	id_nulls	product_name_len	product_name_nulls	price_len	price_nulls	colour_len	colour_nulls	weight_len	weight_nulls	warehouse_id_len	warehouse_id_nulls
3	0	29	0	6	0	7	0	3	0	5	0

Result 67

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:55:18	SELECT MAX(CHAR_LENGTH(id)) AS id_len, SUM(CASE WHEN id IS NULL OR id = '' THEN 1 ELSE 0 EN...	1 row(s) returned	0.000 sec / 0.000 s

```sql
ALTER TABLE products
    MODIFY COLUMN id INT PRIMARY KEY NOT NULL,
    MODIFY COLUMN product_name VARCHAR (40),
    MODIFY COLUMN price DECIMAL (10,2),
    MODIFY COLUMN colour VARCHAR (10),
    MODIFY COLUMN weight DECIMAL (10,2),
    MODIFY COLUMN warehouse_id VARCHAR (5);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:19:44	ALTER TABLE products MODIFY COLUMN id INT PRIMARY KEY NOT NULL, MODIFY COLUMN product_name VARCHAR (40), MODIFY COLUM...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.172 sec

Para relacionar la tabla products con transactions se necesita una tabla intermedia, dado que el campo product_ids de transacciones contiene múltiples valores separados por comas, incumpliendo la primera forma normal, y así resolver la relación muchos a muchos entre transactions y products.
Se crea la tabla transaction_product con los campos: transaction_id, product_id y declined. Se añaden la FK para relacionarla con la tabla products mediante product_id y con la tabla transaction mediante transaction_id.
Para separar los valores del campo product_ids se utiliza JSON-TABLE que permite transformar un array en múltiples filas. Para ello, eliminamos los espacios entre los valores, los concatenamos y los encerramos con [ ].

```sql
CREATE TABLE IF NOT EXISTS transaction_product (
    transaction_id VARCHAR (45),
    product_id INT,
    declined TINYINT,
    PRIMARY KEY (transaction_id, product_id),
    CONSTRAINT fk_transaction_product_products FOREIGN KEY (product_id) REFERENCES products(id),
    CONSTRAINT fk_transaction_product_transactions FOREIGN KEY (transaction_id) REFERENCES transactions (id)
    );

INSERT INTO transaction_product (transaction_id, declined, product_id)
SELECT
    t.id,
    t.declined,
    jt.product_id
FROM transactions t,
JSON_TABLE(
    CONCAT('[', REPLACE(t.product_ids, ', ', ','), ']'),
    '$[*]' COLUMNS (
        product_id INT PATH '$'
    )
) AS jt;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:20:40	CREATE TABLE IF NOT EXISTS transaction_product (transaction_id VARCHAR (45), product_id INT, declined TINYINT, PRIMARY KEY (tra...	0 row(s) affected	0.078 sec
2	19:20:44	INSERT INTO transaction_product (transaction_id, declined, product_id) SELECT t.id, t.declined, jt.product_id FROM transactions t, JSON_...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	6.438 sec

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```sql
SELECT  product_id, product_name, COUNT(product_id) AS total_ventas
FROM transaction_product tp
JOIN products p
ON tp.product_id = p.id
WHERE declined = 0
GROUP BY product_id, product_name
ORDER BY product_id;
```

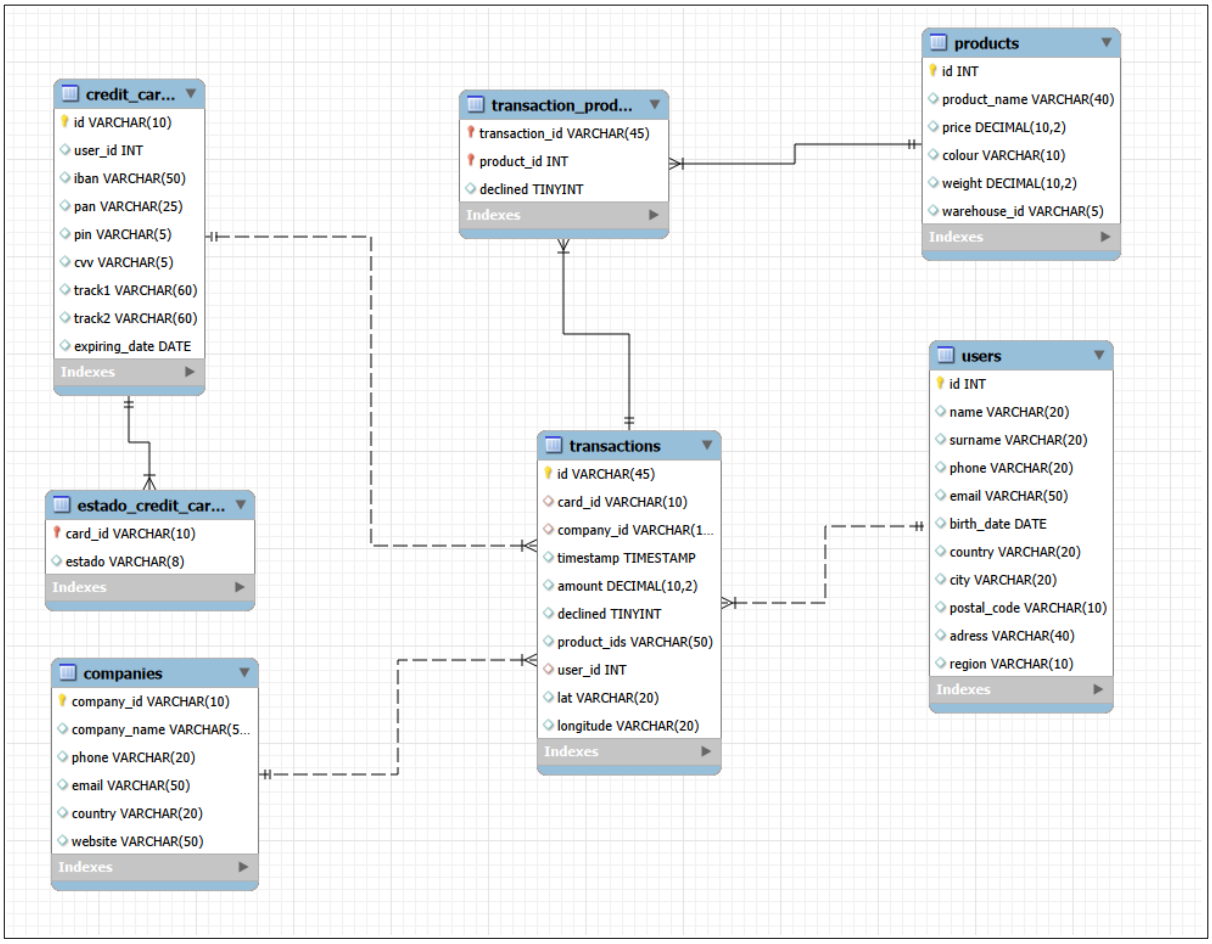
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	total_ventas
1	Direwolf Stannis	2467
2	Tarly Stark	2562
3	duel tourney Lannister	2520
4	warden south duel	2573
5	skywalker ewok	2543

Result 74

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:21:58	SELECT product_id, product_name, COUNT(product_id) AS total_ventas FROM transaction_product tp JOIN products p ON tp.product_id = p.id WH...	100 row(s) returned	1.157 sec / 0.000 s

Con la incorporación de las nuevas tablas (estado_credit_card, products y transaction_product), el modelo en estrella deja de ser completamente puro. Inicialmente se consideró que el nuevo modelo pasaba a ser un esquema en copo de nieve, pero, tras debatirlo en el P2P, se conclyó que sigue siendo un modelo en estrella no puro, ya que al añadir una tabla intermedia (transaction_product) para resolver una relación muchos a muchos, se pierde la relación directa y única entre la tabla de hechos y las dimensiones.